

Όνοματεπώνυμο: Mohammad Matin Marzie Α.Μ.: inf2022001
Εξάμηνο: ΣΤ Ημ/νία: 30 Ιουν 2025

Κοπή Ράβδου (Δυναμικό Προγραμματισμός)

1. Περιγραφή Προβλήματος

Υποθέστε ότι έχουμε μια ράβδο μήκους n και έναν πίνακα $price$ όπου $price[0]$, $price[1]$, $price[2]$, ..., $price[n]$ είναι η τιμή πώλησης ενός κομματιού μήκους i . Ζητείται λοιπόν το μέγιστο ποσό(κέρδος) που μπορεί να προκύψει από την κοπή της ράβδου σε τμήματα μήκους i , για $i = 1, \dots, n$ και την πώληση των κομματιών αυτών. Σημειώνεται ότι το μέγιστο κέρδος μπορεί να προκύψει και χωρίς καμία κοπή, δηλαδή από την πώληση της ράβδου ως ενιαίο κομμάτι (πχ. Όταν $\max\{price[i]\} = price[n]$ [1, σελ.358]).

2. Περιγραφή Αλγορίθμου/Μεθόδου

Για την επίλυση του προβλήματος εφαρμόζουμε την τεχνική του **Δυναμικού Προγραμματισμού**, την οποία παρουσίασε πρώτος ο Richard Bellman τη δεκαετία του 1950 [2, Bellman].

Ως τέτοια, ορίζουμε τη **δομή** της βέλτιστης λύσης και την **αναδρομική σχέση της τιμής** της εν λόγω λύσης. Αν κάνουμε μια πρώτη κοπή στο μήκος i , τότε:

- το πρώτο κομμάτι έχει μήκος i , αξία $price[i]$,
- απομένει ράβδος μήκους $n-i$,

για την οποία βρίσκουμε τη βέλτιστη λύση. Άρα η βέλτιστη λύση για n δίνεται από:

$$optimal(n) = \max\{ price[i] + optimal(n-i) \}$$

Όπου $optimal(n)$ είναι το μέγιστο κέρδος για μήκος n [3, Dekhtyar].

Η προαναφερθείσα αναδρομική σχέση χρησιμοποιεί τη **βέλτιστη υποδομή(optimal substructure)** υπολογίζοντας τη μέγιστη τιμή για κάθε μήκος από τις βέλτιστες λύσεις μικρότερων μηκών, επαναχρησιμοποιώντας τις.

Κατόπιν του ορισμού της αναδρομικής σχέσης, περνάμε στο τρίτο βήμα [1, σελ. 357]:

τον **υπολογισμό** της τιμής της βέλτιστης λύσης, εργαζόμενοι κατά κανόνα **Αναβιβαστικά (bottom-up)** και επανειλημμένα (**iteratively**).

Εφαρμόζοντας λοιπόν την τεχνική Δ.Π. μειώνουμε σημαντικά την χρονική και την χωρική πολυπλοκότητα από $O(2^{n-1})$ και $\approx O(2^n)$ σε $O(n^2)$ και $O(n)$ αντίστοιχα.

3. Υλοποίηση

Η υλοποίηση έγινε στη γλώσσα Python, βασισμένη στον ψευδοκώδικα που παρουσιάζεται στην [1, σελ. 366]. Αναπτύχθηκε η συνάρτηση $RodCutting(n, price)$, η οποία υπολογίζει το μέγιστο κέρδος καθώς και το αντίστοιχο σχέδιο κοπής για κάθε μήκος ράβδου από 1 έως n .

Επιπλέον, δημιουργήθηκε παραμετροποιημένη εκδοχή $RodCutting_1_3_4(n, price)$, στην οποία επιτρέπονται μόνο συγκεκριμένα μήκη κοπής (1, 3 και 4), για την αντιμετώπιση περιπτώσεων με περιορισμούς στα διαθέσιμα μήκη.

Εξάλλου εκτυπώνεται στον τερματικό τα παραχθέντα αποτελέσματα του προγράμματος όπως φαίνεται στην παρακάτω εικόνα:

```

lilsbib@matin:~/univ-4/algorithms/inf2022001-Κοπή-Ράβδου$ python3 main.py
Can sell with maximum profit of: 12
Needed cuts: [1, 2, 2]
Can sell with maximum profit of: 11
Needed cuts: [1, 1, 3]
lilsbib@matin:~/univ-4/algorithms/inf2022001-Κοπή-Ράβδου$

```

Figure 1: Αποτελέσματα

* Καθ'όλη τη διάρκεια της υλοποίησης δεν χρησιμοποιήθηκε έτοιμος κώδικας από εξωτερικές πηγές.

4. Παράδειγμα

Όπως αναφέρθηκε στην [2. Περιγραφή Αλγορίθμου/Μεθόδου](#) χρησιμοποιείται η τεχνική bottom-up και η αναδρομική σχέση $optimal(n)$. Ως εκ τούτου στον παρακάτω πίνακα υπολογίζουμε τις βέλτιστες λύσεις $optimal(0 \dots 5)$ για τα μήκη (length) από 1 έως length δηλαδή ($n=5$ με τιμές $p = [2,5,7,8,10]$)

Length 0...n	$optimal(length) = price[length] + optimal(n-length)$	Max κόστος /λύση	optimal_table Best Prices	Cuts
0	-	0	[0, 0, 0, 0, 0, 0]	-
1	price[1-1] + optimal_table[0] = 2	2	[0, 2, 0, 0, 0, 0]	[1]
2	price[1-1] + optimal_table[1] = 4 price[2-1] + optimal_table[0] = 5	5	[0, 2, 5, 0, 0, 0]	[2]
3	price[1-1] + optimal_table [2] = 7 price[2-1] + optimal_table [1] = 7 price[3-1] + optimal_table [0] = 7	7	[0, 2, 5, 7, 0, 0]	[2, 1]
4	price[1-1] + optimal_table [3] = 9 price[2-1] + optimal_table [2] = 10 price[3-1] + optimal_table [1] = 9 price[4-1] + optimal_table [0] = 8	10	[0, 2, 5, 7, 10, 0]	[2, 2]

5	price[1-1] + optimal_table[4] = 12 price[2-1] + optimal_table[3] = 12 price[3-1] + optimal_table[2] = 12 price[4-1] + optimal_table[1] = 10 price[5-1] + optimal_table[0] = 10	12	[0, 2, 5, 7, 10, 20]	[1, 2, 2]
---	---	----	----------------------	-----------

5. Συμπεράσματα

Ο αλγόριθμος Δυναμικού Προγραμματισμού για την κοπή ράβδου είναι αποτελεσματικός και επεκτάσιμος καθώς μειώνει την πολυπλοκότητα από εκθετικό σε πολυωνυμικό βαθμό. Παρέχει τη βέλτιστη λύση αξιοποιώντας βέλτιστες υποδομές και επαναχρησιμοποίηση ενδιάμεσων υπολογισμών. Περιορίζεται όμως σε διακριτές τιμές κοπών. Μελλοντικά μπορεί να υποστηρίξει συνεχή μήκη ή περισσότερα κριτήρια βελτιστοποίησης.

Βιβλιογραφία

[1] Thomas H. Cormen et al. “Εισαγωγή στους Αλγορίθμους”, 1η Έκδοση. ΙΤΕ-Πανεπιστημιακές εκδόσεις κρίτης, 2016. ISBN: 978-960-524-473-6 [[ιστοσελίδα](#)]

[2] Bellman, Richard. "The theory of dynamic programming." Bulletin of the American Mathematical Society 60.6 (1954): 503-515 [[scholar](#)]

[3] Alexander Dekhtyar. “Design and Analysis of Algorithms”. [[pdf](#)]